

Дисциплина	Технологии индустриального программирования
Институт	Перспективных технологий и индустриального программирования
Кафедра	Индустриального программирования
Вид учебного материала	Практические занятия
Преподаватель	Микерина Анна Дмитриевна
Семестр	3 семестр, 2025-2026

Практическое занятие №6

Практическое занятие №6 по дисциплине «Технологии индустриального программирования» посвящено углублённой работе с DOM (Document Object Model) и управлению размерами, стилями и прокруткой элементов веб-страницы с использованием JavaScript. С теоретической базой можно ознакомиться в репозитории: <https://github.com/dv0retsky/js-tutorial> (главы 20–25).

В ходе практики вы освоите методы поиска и изменения элементов DOM, управления стилями и классами, а также работы с геометрическими свойствами (размеры, прокрутка) элементов и окна браузера. Итогом занятия станет создание сценариев для динамического управления содержимым, стилизацией и прокруткой страницы, что позволит разрабатывать интерактивные и адаптивные веб-интерфейсы, включая формы с динамическим поведением.

Задачи для самостоятельной отработки

Задание №1.1 Поиск элемента и изменение содержимого

Напишите функцию `updateMessage(id, message)`, которая находит элемент по `id` и заменяет его содержимое на переданный текст `message` с помощью `innerHTML`.

Используйте `document.getElementById`. Проверьте, как `innerHTML` обрабатывает HTML-теги (например, `message = '<strong>Новое сообщение!</strong>'`).

```
<div id="message">Старое сообщение</div>
<script>
  function updateMessage(id, message) {
    // Ваш код
  }
  updateMessage('message', 'Новое сообщение!');
</script>
```

Задание №1.2 Перемещение по DOM-дереву

Напишите функцию `highlightSibling(id)`, которая находит элемент по `id`, получает его следующего соседа (`nextSibling`) и добавляет ему текст `<strong>Выделено</strong>` через `innerHTML`.

Учтите, что `nextSibling` может возвращать текстовый узел. Используйте `nextElementSibling` для надёжности.

```
<ul>
  <li id="target">Элемент 1</li>
  <li>Элемент 2</li>
</ul>
<script>
  function highlightSibling(id) {
    // Ваш код
  }
  highlightSibling('target');
</script>
```

Задание №1.3 Список детей элемента

Напишите функцию `listChildren(selector)`, которая принимает CSS-селектор, находит элемент, получает его дочерние элементы через `children` и выводит их `innerHTML` в консоль.

Используйте `document.querySelector` и `children`.

```
<ul id="list">
  <li>Элемент 1</li>
  <li>Элемент 2</li>
  <li>Элемент 3</li>
</ul>
<script>
  function listChildren(selector) {
    // Ваш код
  }
  listChildren('#list');
</script>
```

Задание №2.1 Очистка списка

Создайте функцию `clear(elem)`, которая удаляет всё содержимое из элемента `elem`.

Используйте `innerHTML = ''` или цикл с `remove()`. Объясните, какой метод быстрее для больших списков.

```
<ol id="elem">
  <li>Привет</li>
  <li>Мир</li>
</ol>
<script>
  function clear(elem) {
    // Ваш код
  }
  clear(document.getElementById('elem'));
</script>
```

Задание №2.2 Добавление поля формы

Напишите функцию `addInput(formId)`, которая добавляет новое поле `<input>` в форму с указанным `id` через `createElement` и `append`.

Используйте `document.createElement` и `form.append`. Проверьте, как добавление влияет на отправку формы.

```
<form id="myForm">
  <input type="text" value="Поле 1">
</form>
<script>
  function addInput(formId) {
    // Ваш код
  }
  addInput('myForm');
</script>
```

Задание №2.3 Клонирование и замена

Напишите функцию `cloneAndReplace(id)`, которая клонирует элемент по `id` с помощью `cloneNode(true)`, изменяет текст в клоне и заменяет оригинал через `replaceWith`.

Используйте `cloneNode(true)` для глубокого клонирования. Проверьте, сохраняется ли `id` в клоне.

```
<div id="box">Оригинал</div>
<script>
  function cloneAndReplace(id) {
    // Ваш код
  }
  cloneAndReplace('box');
</script>
```

Задание №3.1 Подсветка ошибки

Напишите функцию `highlightError(id)`, которая добавляет класс `error` к элементу по `id` через `classList.add`.

Используйте `classList.add`.

```
<style> .error { border: 2px solid red; } </style>
<input id="input" value="Текст">
<script>
  function highlightError(id) {
    // Ваш код
  }
  highlightError('input');
</script>
```

Задание №3.2 Переключение стиля

Напишите функцию `toggleValid(id)`, которая переключает класс `valid` через `classList.toggle` и изменяет `style.backgroundColor` на `'green'` или `''` в зависимости от наличия класса.

Используйте `classList.toggle` и `classList.contains`. Проверьте сброс стиля через `style.backgroundColor = ''`.

```
<style> .valid { border: 2px solid green; } </style>
<input id="input" value="Текст">
<script>
  function toggleValid(id) {
    // Ваш код
  }
  toggleValid('input');
</script>
```

Задание №3.3 Динамическая ширина поля

Напишите функцию `adjustWidth(id)`, которая читает текущую ширину `<input>` через `getComputedStyle`, увеличивает её на 50px и применяет через `style.width`.

Используйте `getComputedStyle` и `parseInt` для работы с пикселями. Учтите единицы измерения.

```
<style> input { width: 100px; } </style>
<input id="input" value="Текст">
<script>
  function adjustWidth(id) {
    // Ваш код
  }
  adjustWidth('input');
</script>
```

Задание №4.1 Проверка видимости

Напишите функцию `isHidden(id)`, которая проверяет, скрыт ли элемент по `id`, используя `offsetWidth` и `offsetHeight`.

Используйте `offsetWidth` и `offsetHeight`. Проверьте, что возвращает функция для элемента с нулевыми размерами.

```
<div id="box" style="display:none">Скрытый</div>
```

```
<script>
function isHidden(id) {
    // Ваш код
}
alert(isHidden('box')); // true
</script>
```

Задание №4.2 Расширение элемента

Напишите функцию `expandElement(id)`, которая устанавливает высоту элемента равной `scrollHeight` через `style.height`.

Используйте `scrollHeight` и `style.height`. Убедитесь, что добавлены единицы измерения (px).

```
<div id="box" style="height:100px;overflow:auto">
    Длинный текст...
</div>
<script>
function expandElement(id) {
    // Ваш код
}
expandElement('box');
</script>
```

Задание №4.3 Прокрутка к концу

Напишите функцию `scrollToBottom(id)`, которая прокручивает элемент по `id` к его нижней части через `scrollTop`.

Используйте `scrollTop` и `scrollHeight`. Проверьте, как вычислить максимальную прокрутку.

```
<div id="box" style="height:100px;overflow:auto">
    Длинный текст...
</div>
<script>
function scrollToBottom(id) {
    // Ваш код
}
scrollToBottom('box');
</script>
```

Задание №5.1 Отображение размеров окна

Напишите функцию `showWindowSize()`, которая выводит текущую ширину и высоту окна через `document.documentElement.clientWidth/Height`.

Используйте `clientWidth/Height`.

```
<script>
function showWindowSize() {
  // Ваш код
}
showWindowSize();
</script>
```

Задание №5.2 Плавная прокрутка вниз

Напишите функцию `smoothScrollDown()`, которая прокручивает страницу на 500px вниз с плавной анимацией через `scrollBy`.

Используйте `window.scrollBy` с `{ behavior: 'smooth' }`.

```
<div style="height:2000px">Длинная страница</div>
<script>
function smoothScrollDown() {
  // Ваш код
}
smoothScrollDown();
</script>
```

Задание №5.3 Прокрутка к элементу

Напишите функцию `scrollToElement(id)`, которая прокручивает страницу к элементу по `id` с плавной анимацией через `scrollIntoView`.

*Используйте `scrollIntoView({ behavior: 'smooth', block: 'start' })`.
Проверьте параметры `options`.*

```
<div style="height:1000px">Верх</div>
<div id="target">Цель</div>
<div style="height:1000px">Низ</div>
<script>
function scrollToElement(id) {
  // Ваш код
}
scrollToElement('target');
</script>
```


Дополнительные задачи для самостоятельной отработки

Задание №6 Динамическая форма с валидацией

Создайте форму с полем `<input>` и кнопкой "Добавить". При клике на кнопку добавляется новое поле `<input>` с классом `valid` или `error` в зависимости от введенного текста (не пустое — `valid`, пустое — `error`). Если ширина формы превышает 500px, она становится прокручиваемой (`overflow: auto`).

Используйте методы из тем №22 (создание/вставка), №23 (стили/классы), №24 (размеры). Связь с событиями через обработчик `onclick`.

```
<style>
  .valid { border: 2px solid green; }
  .error { border: 2px solid red; }
</style>
<form id="myForm">
  <input type="text">
  <button type="button" onclick="addField()">Добавить</button>
</form>
<script>
  function addField() {
    // Ваш код
  }
</script>
```

Задание №7 Прокручиваемый список с подсветкой

Создайте список `<ul>` с 10 элементами `<li>`. При клике на элемент он получает класс `highlight`, а страница плавно прокручивается к нему через `scrollIntoView`. Если элемент уже выделен, класс снимается, и прокрутка возвращается в начало через `scrollTo`.

Используйте темы №20 (поиск элементов), №23 (стили), №25 (прокрутка). Добавьте обработчик событий.

```
<style>
  .highlight { background-color: yellow; }
  ul { height: 200px; overflow: auto; }
</style>
<ul id="list">
  <li>Элемент 1</li>
  <!-- ... до Элемент 10 -->
</ul>
```

```
<script>
  // Ваш код
</script>
```

Задание №8 Адаптивная форма с динамической шириной

Создайте форму с `<input>` и кнопкой "Адаптировать". При клике на кнопку ширина `<input>` устанавливается равной текущей ширине окна (`clientWidth`), но не больше 80% от `offsetWidth` формы, с учётом полосы прокрутки. Если поле становится шире формы, форма становится прокручиваемой.

Используйте темы №23 (стили, `getComputedStyle`), №24 (размеры), №25 (размеры окна). Учтите единицы измерения.

```
<style>
  form { width: 400px; border: 1px solid black; }
</style>
<form id="myForm">
  <input id="input" type="text">
  <button type="button" onclick="adaptWidth()">Адаптировать</button>
</form>
<script>
  function adaptWidth() {
    // Ваш код
  }
</script>
```